Defense:
Prevent, Mitigate, Respond & Test

Taller de Seguridad

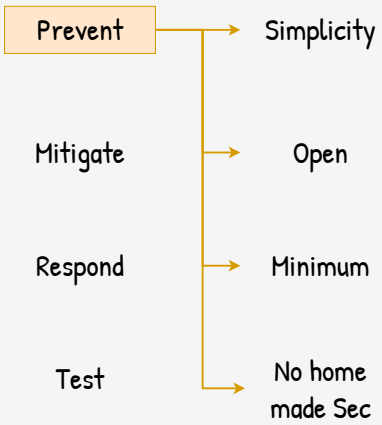
Ing. Martin Di Paola - Fiuba

Prevent

Mitigate

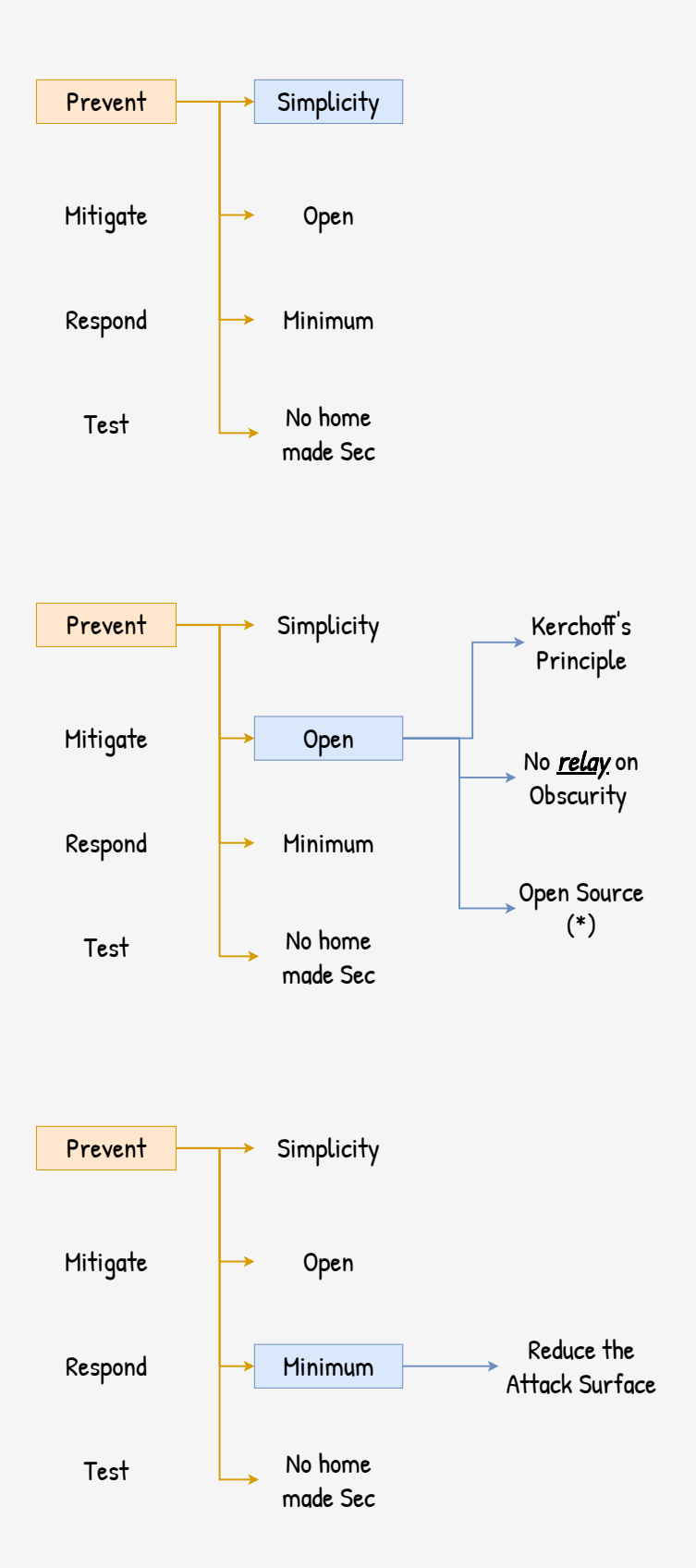
Respond

Test



En el plano defensivo tenemos a 4 pilares: prevención, mitigación, respuesta y testeo. No es una taxonomia completa, pero funciona bastante bien como guía o blueprint para diseñar e implementar mecanismos de defensa.

Prevenir es mejor que curar. Frase milenaria totalmente vigente. En todo momento, si se puede evitar, hacerlo. Recuperarse tras un hack va a ser mucho mas costoso.



Un sistema difícil de usar deja margen para errores:

```
- difícil de configurar -> se termina dejando agujeros

- difícil / tedioso de usar -> los users van a ser los

  los mecanismos de seguridad
```

Pero simplicidad no se refiere solamente a "fácil de usar" sino a que el sistema en si sea tambien simple.

Un sistema complejo tiene demasiadas "piezas moviles" y las posibles interacciones internas hacen q sea facil pasarse por alto alguna vulnerabilidad.

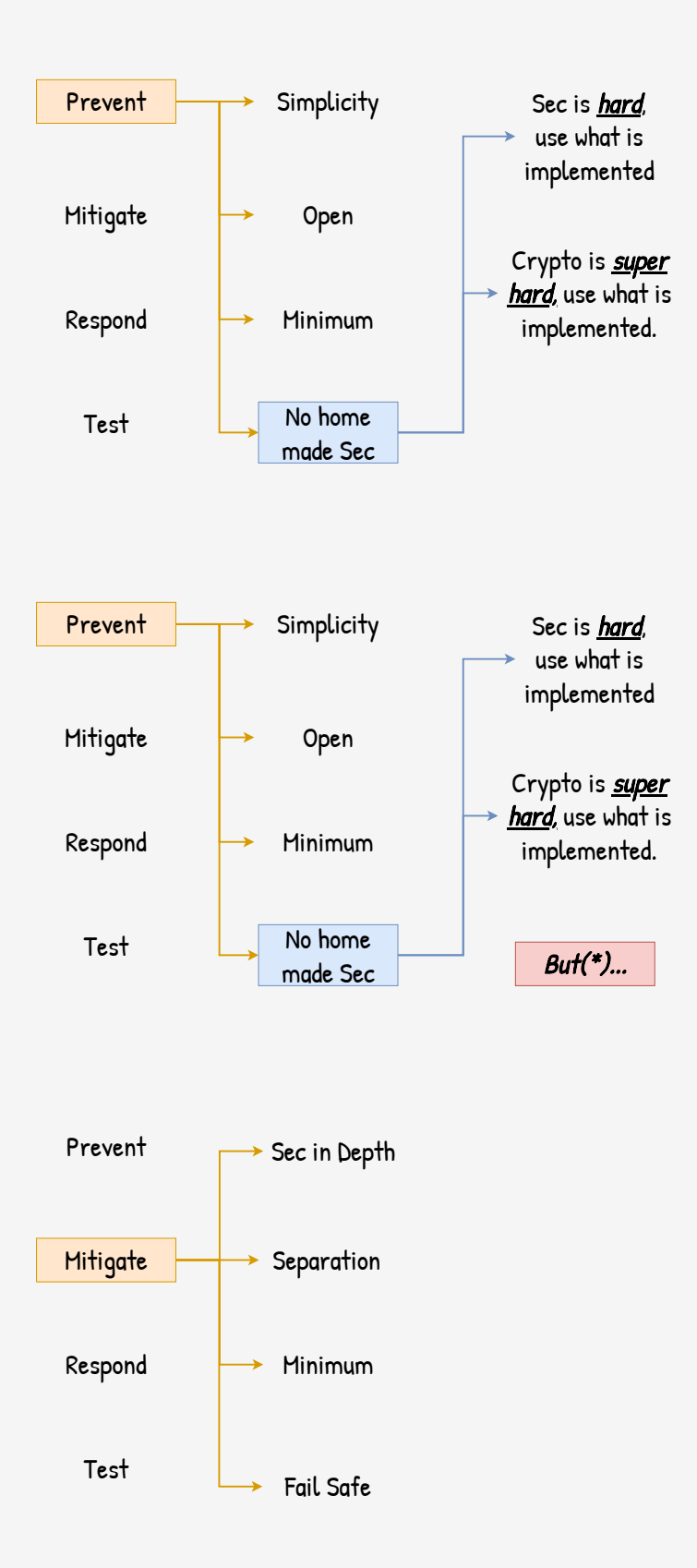
Principio de Kerckhoff: un sistema criptografico debe ser seguro aun si el adversario sabe absolutamente todo exception la clave secreta.

El principio puede extrapolarse a sistemas de seguridad en genera: un sistema debe ser seguro aun si se conoce absolutamente todo de él. Ocultar detalles de implementación como única protección es totalmente frágil y es cuestion de tiempo hasta q alguien reversee y venga con un ataque.

Open source rules.

El único asterisco es que en ciertas situaciones ocultar detalles potencia la defensa (pero nunca debe ser la única).

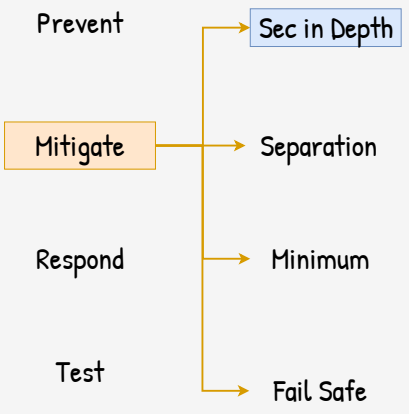
Alineado con Simplicidad, un sistema lo mas chico posible reduce el area expuesta a problemas.



Implementar mecanismos de seguridad es difícil, implementar es más difícil aun. Siempre usar implementaciones estandar y no salir a implementar cosas custom.

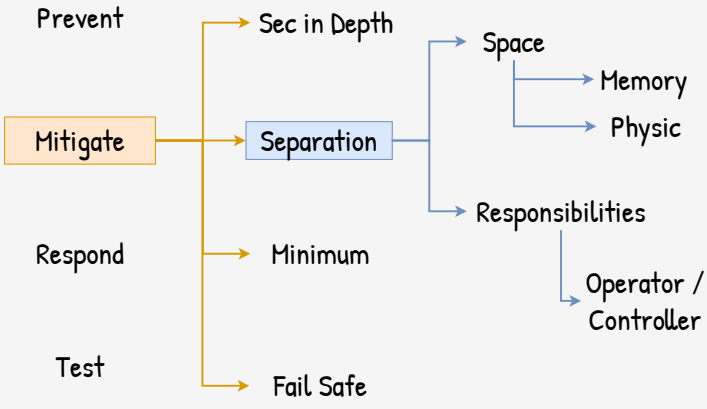
Dicho eso, ciertas situaciones requieren implementaciones hechas a medida.

Si no podes evitar el hack, hace que tenga un menor impacto / daño.



Seguridad en lo profundo. Checks a lo largo de toda la cadena de procesamiento.

Si un adversario logro llegar al servidor productivo, q no pueda llegar a la DB. Tal vez tire prod por unas horas, pero al menos no se llevo la data de los clientes.



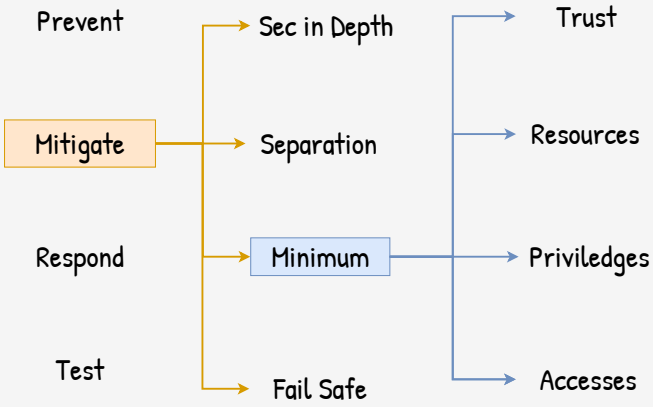
Separar / aislar: cuando hay un hack queremos que el adversario no pueda moverse hacia otras areas / sistemas / procesos.

Si 2 sistemas estan conviviendo en la misma area, es probable q el adversario pueda infectar uno desde el otro.

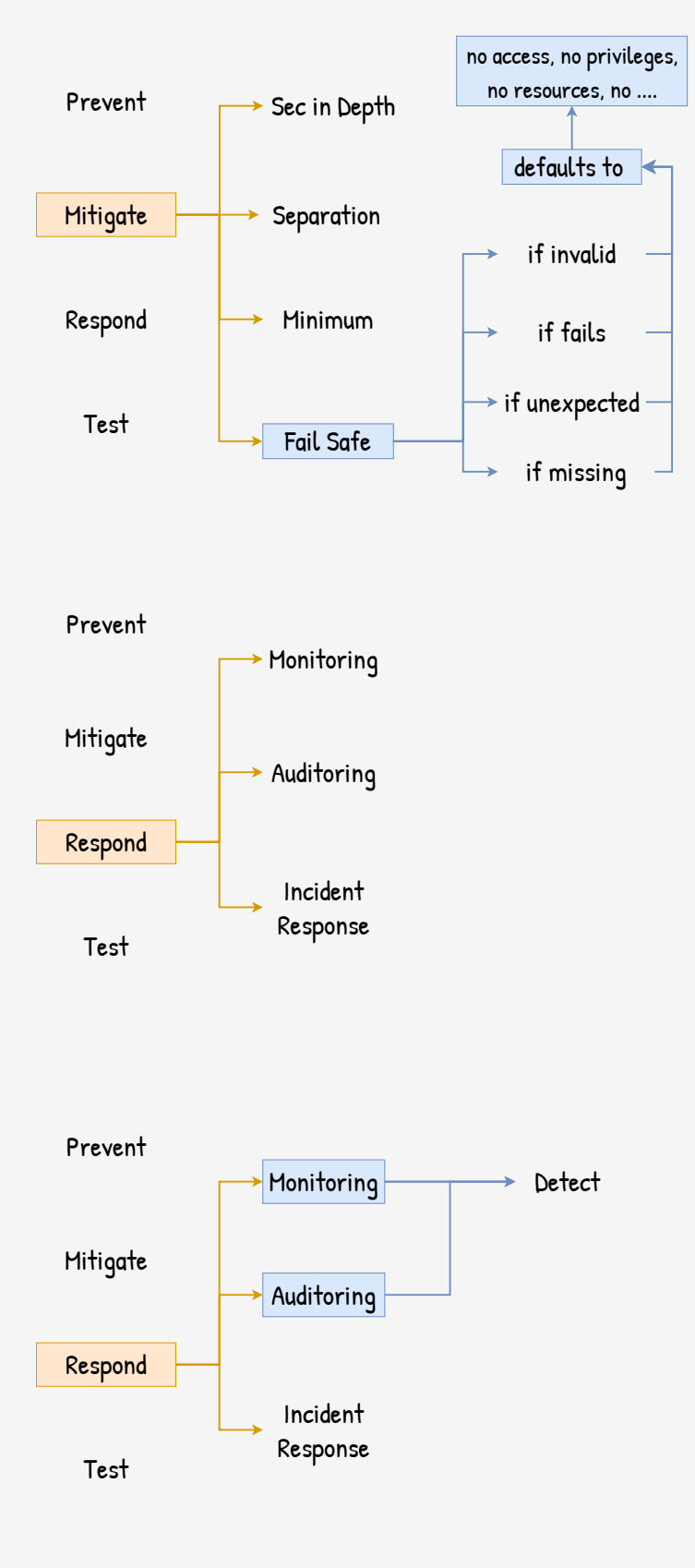
Si dichos sistemas estan totalmente aislados, el adversario estara en un callejon sin salida.

En la práctica, aislación total no es posible, pero cuanto más se pueda aislar, mejor.

Y separación no es solo a nivel software/hardware, es tambien a nivel humano.



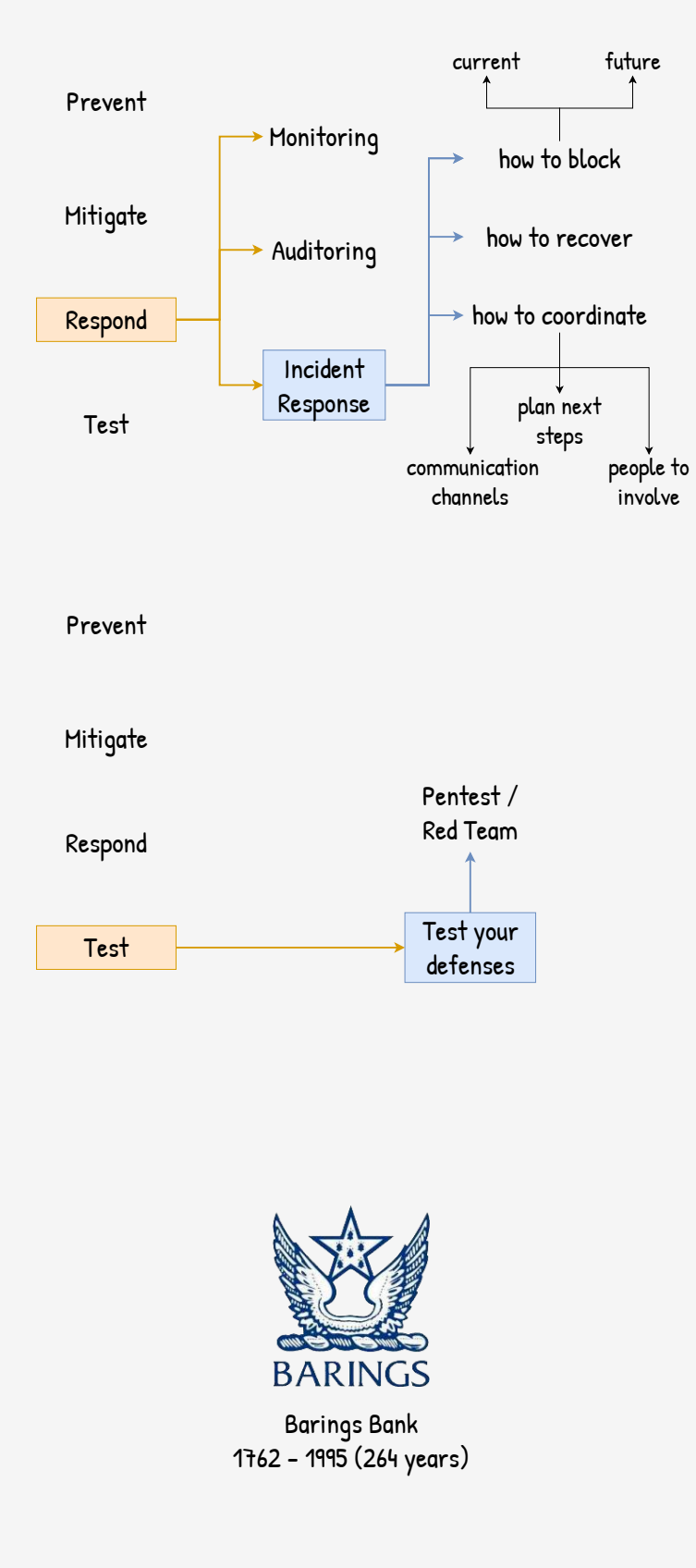
Si una cuenta es hackeada, el adversario tendra menos poder si la cuenta tiene la minima cantidad de recursos y accesos.



Cuando algo salga mal, el sistema deberia defaultear a un estado de minimo privilegio, un estado "seguro".

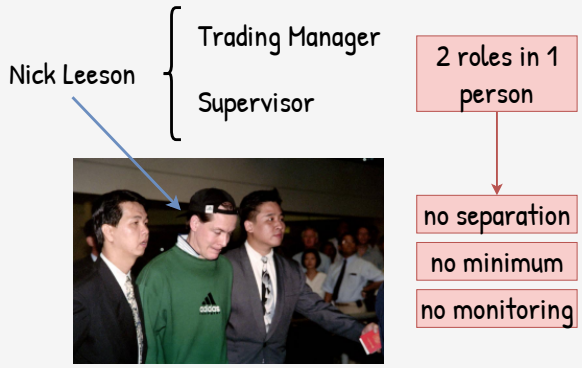
.

Monitoreo y auditoría son 2 caras de una misma moneda. Monitoreo (+alertas) es para la deteccion temprana; Auditoría es para la detección o análisis forence de algun evento de seguridad que ya pasó.



Shit happens. Y cuando pase, cuales son los next steps? Tener un plan, saber a quien llamar y como encarar un "incidente" en curso hace que se tengan respuestas más rápidas y minimizan el daño.

No importa que tan bien hayas puesto las defensas. Hay q ponerlas a prueba.



Cual es el problema que una persona sea el encargado de hacer trades y de ser su propio supervisor?

No hay separacion de roles, no hay privilegios minimos, no hay monitoreo.

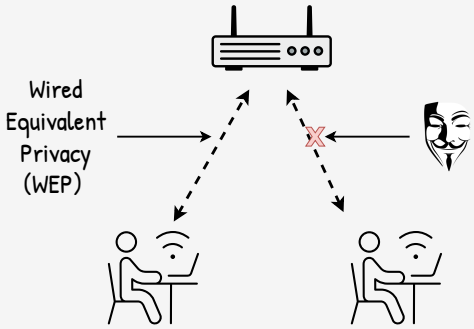
Es *poner al zorro a cuidar del gallinero.*

El resultado?

El resultado? Un banco de más de 260 años de antiguedad, donde la Reina de Inglaterra tenía cuenta, terminó en banca rota y siendo vendido por 1 libra esterlina.

La foto corresponde a Nick Leeson siendo capturado en Kuala Lumpur tras casi 1 año de estar prófugo.

(fuente: Washington Street Journal)



Con la aparición de la wifi se implementaron mecanismos de encriptación. El primero fue WEP - Wired Equivalent Privacy.

WEP 1997

(closed-ish) (impl)

El problema? Código cerrado. No se sabia como estaba hecho.

WEP 1997 xx-2001

(closed-ish) (impl) (RC4 confirmed)

Hasta que se confirmo (via research) q se estaba usando RC4 como stream cipher.

WEP 1997 xx-2001

(closed-ish) (impl) (RC4 confirmed)

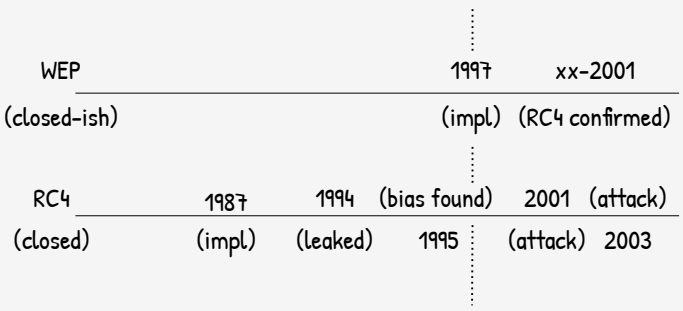
RC4 1987 1994 (bias found)

(closed) (impl) (leaked) 1995

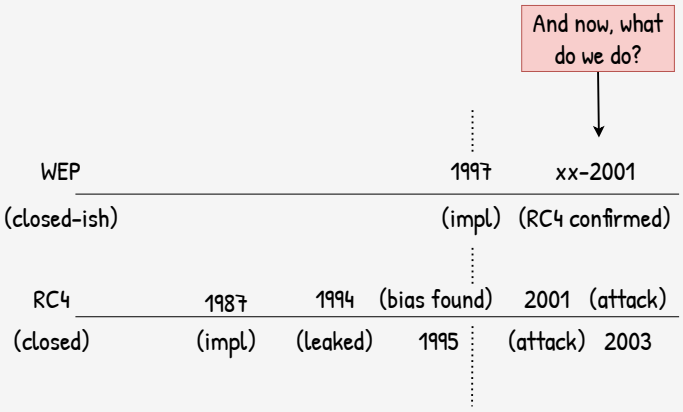
RC4 es un stream cipher viejo, cuyo codigo fue leakeado en los 90s y q para ~1995 ya se sabia que habia algunos biases.

Para el ~2000 los biases se transformaron en ataques completos. Un atacante podia en un par de horas romper la cripto completamente, abusandose de las vulnerabilidades en el RC4 como en el protocolo WEP.

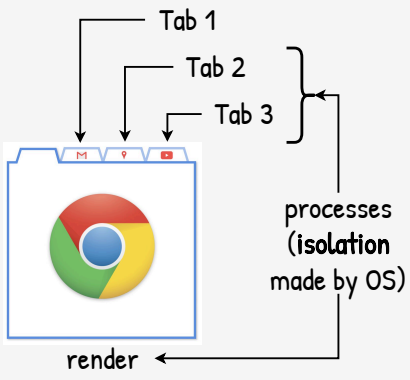
Desarrollar in-house cripto sin el correcto disclosure hizo que WEP sea vulnerable y que nos dieramos cuenta años luego de q habia miles de dispositivos ya vendidos.

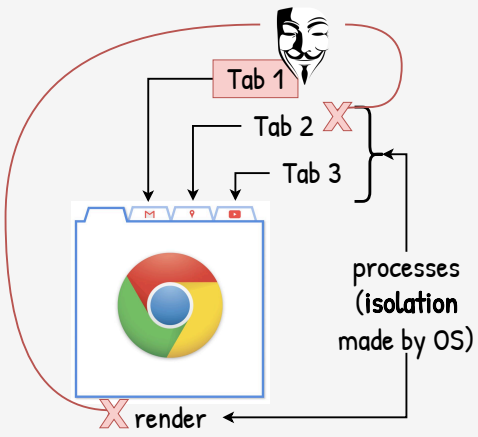


.



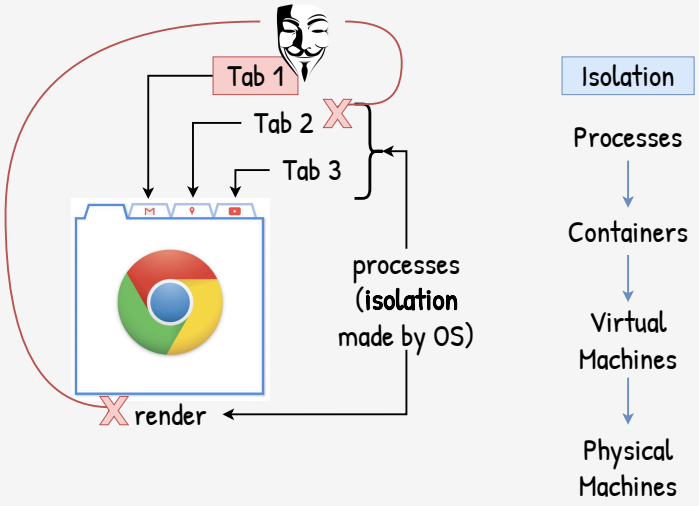
Cuando uno abre Chrome en realidad hay 1 proceso encargado de dibujar en pantalla (el render) y un proceso por cada Tab abierta.



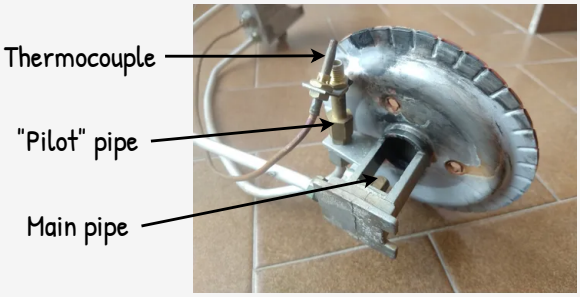


Al ser procesos, un adversario q haya tomado el control de una Tab (y del proceso), no tiene acceso ni a otras Tabs ni al render.

Es el OS que fuerza la separación entre procesos por lo que el adversario debera o bypassear al OS o encontrar algun bug en la comunicación entre las Tabs y el render. No es imposible, pero es mucho más difícil.

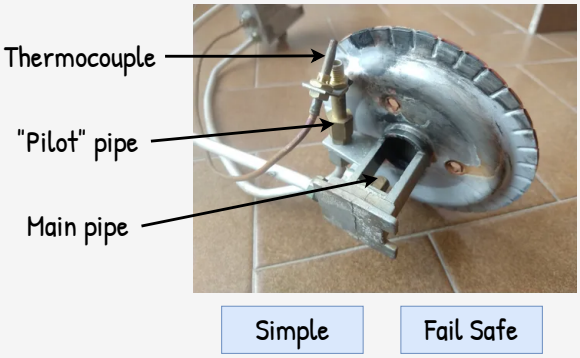


La idea puede profundizarse mucho más: procesos < containers < vms < physical devices



La termocupla controla el paso de gas de las cañerias principal y piloto. Cuando el sistema esta apagado, la termocupla esta fria y corta el suministro de gas.

Para encenderlo hay q manualmente habilitar el paso de gas, prender el piloto y esperar a q la termocupla se caliente para q de paso libre por si solo.

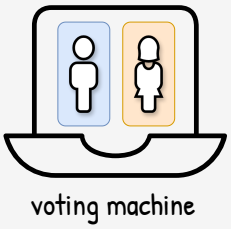


Q pasa si se corta el gas? Q pasa si se apaga el piloto?

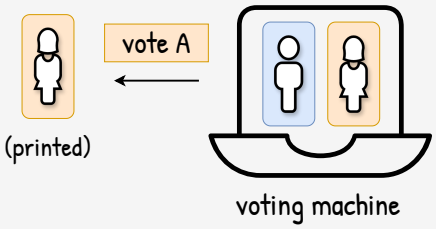
No hay peligro. A pesar de que la válvula de gas este abierta, al apagarse el piloto la termocupla se enfria y cierra el suministro de gas.

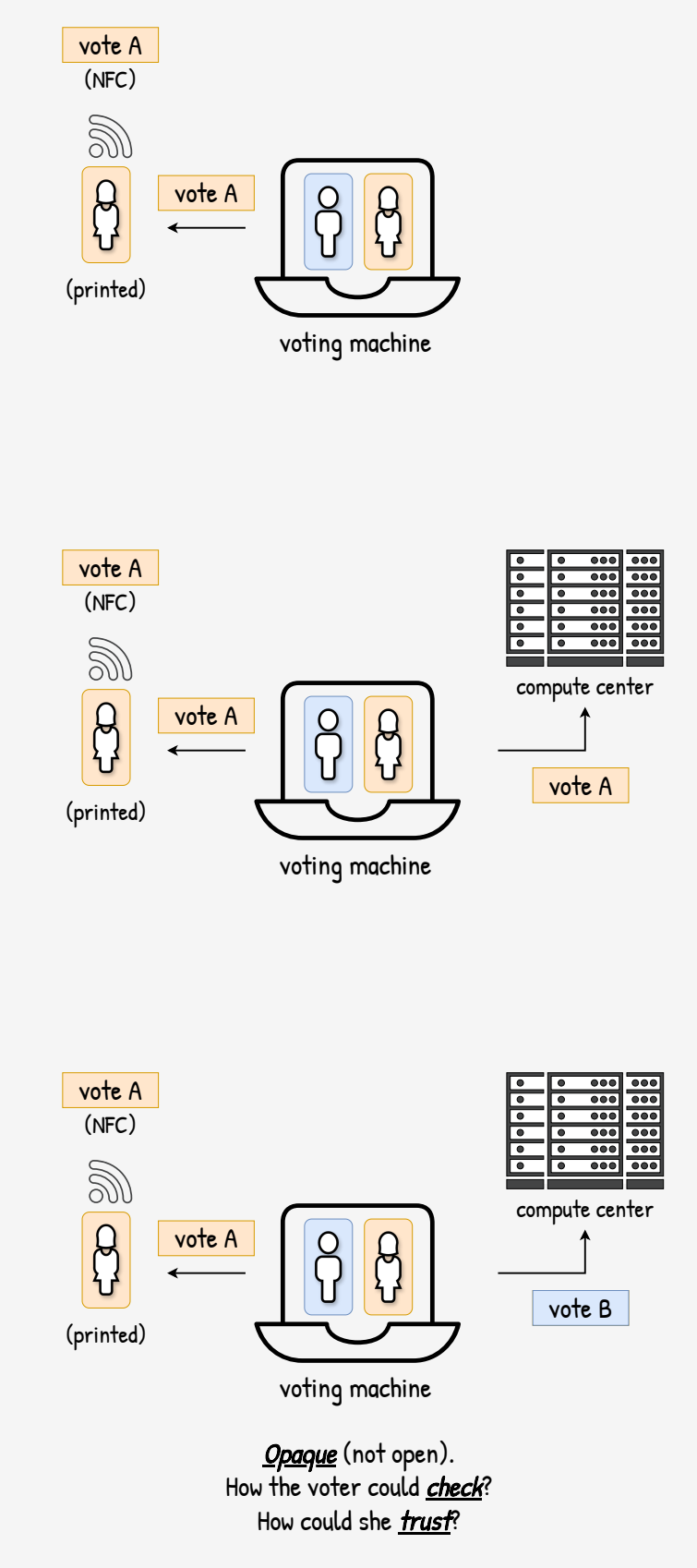
El sistema falló pero de forma segura.

.



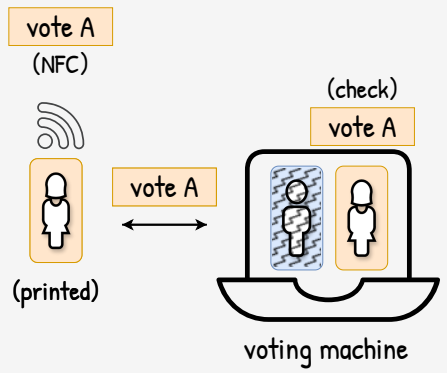
.



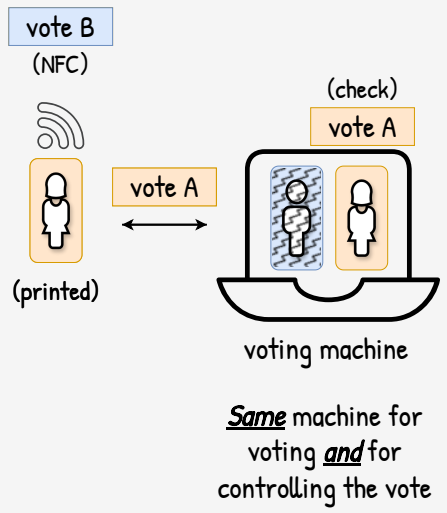


Al votar por A se imprime el voto con un NFC q indica que se voto a A. Al mismo tiempo se envia un mensaje al centro de votación diciendo el resultado del voto.

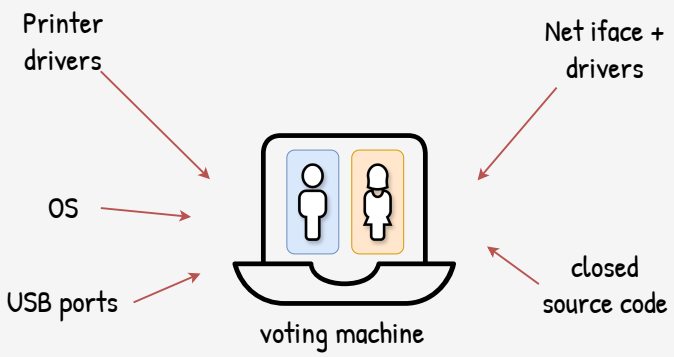
Pero el sistema es cerrado. Quien puede verificar q en el momento de la votacion el voto a A no sea cambiado por B?



El votante puede verificar que el voto impreso es efectivamente un voto por A acercando la boleta a la máquina de votación para q esta lea el NFC.



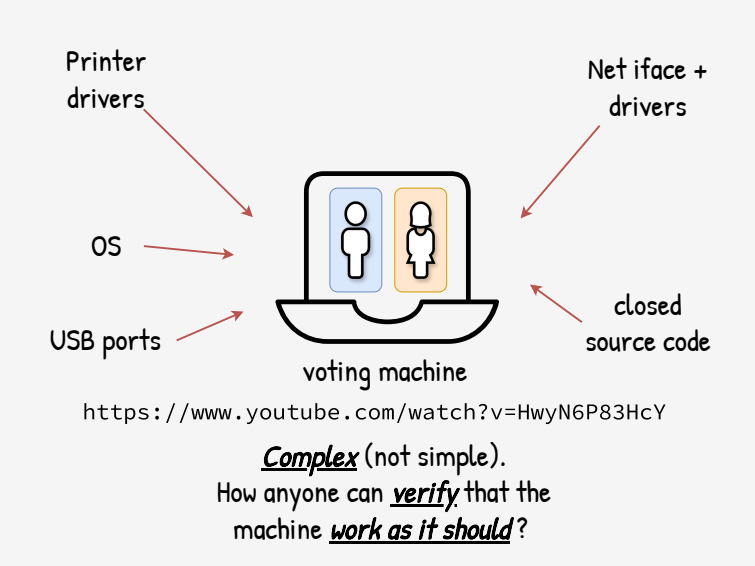
Pero si modelamos q la máquina es adversarial y nos imprimió un voto para A que en realidad es para B, no podemos confiar en que nos va a hacer la verificación honestamente!



https://www.youtube.com/watch?v=HwyN6P83HcY

La máquina de votación es mucho más que una mera impresora de boletas.

https://www.youtube.com/watch?v=HwyN6P83HcY



Es demasiado complejo. Puede fallar de múltiples formas, expone una superficie grande para q pueda ser hackeada desde varios ángulos y simplemente no es transparente.